

第 6 週：狀態機設計

本週一句話：需要「記住現在進行到哪一步」的電路，就用狀態機。

本週指令

雙擊 `env.bat` 開好環境（提示字元要有 `[OSS CAD Suite]`），然後：

```
cd week06_fsm
```

每個範例都是這三條（外加一條檢查），把 `03_debounce` 換成你要跑的名字：

```
iverilog -o sim.out 03_debounce.v 03_debounce_tb.v
echo %ERRORLEVEL%
vvp sim.out
gtkwave 03_debounce.gtkw
```

`echo %ERRORLEVEL%` 印 0 才是編譯成功——失敗時 iverilog 常常一個字都不印。

本週範例名稱：

- `01_traffic_moore`
- `02_seq_detect`
- `03_debounce`
- `04_vending`

下面每個範例的段落，都直接附了它自己的四行指令。

學完你要會什麼

- [] ★★ 會寫三段式狀態機（業界標準）

- [] 分得出 Moore 和 Mealy
- [] 知道為什麼一定要有 `default`
- [] 會畫狀態圖再翻成程式碼
- [] 會用狀態機解實際問題 (去彈跳、序列偵測、販賣機)

一、★★ 三段式寫法 (背起來)

```
// ===== 第 1 段：狀態暫存器 (時序，用 <=) =====
always @(posedge clk) begin
    if (rst) state <= S_IDLE;
    else     state <= next_state;
end

// ===== 第 2 段：下一狀態邏輯 (組合，用 =) =====
always @(*) begin
    next_state = state;           // ★ 預設留在原狀態，避免 latch
    case (state)
        S_IDLE: if (start) next_state = S_RUN;
        S_RUN:  if (done)  next_state = S_IDLE;
        default: next_state = S_IDLE;
    endcase
end

// ===== 第 3 段：輸出邏輯 (組合，用 =) =====
always @(*) begin
    case (state)
        S_IDLE: busy = 1'b0;
        S_RUN:  busy = 1'b1;
        default: busy = 1'b0;
    endcase
end
```

為什麼要分三段？

好處	說明
狀態轉移一目瞭然	第 2 段就是狀態圖的翻譯，可以直接對照
不會誤產生 latch	第 2、3 段都有預設值和 default
輸出乾淨	輸出從組合邏輯出來，時序好分析
好維護	加一個狀態只要改兩個 case，不會動到別的

[!WARNING] **default** 一定要寫。兩個理由：

1. 沒寫 → 產生 latch (第 3 週的坑)
2. 真實電路遇到雜訊可能跑到未定義狀態 → **default** 把它救回 IDLE

二、Moore vs Mealy

	Moore	Mealy
輸出看什麼	只看現在的狀態	看狀態 + 輸入
反應速度	慢一拍	即時
輸出品質	乾淨穩定	會跟著輸入抖動 (可能有毛刺)
需要的狀態數	通常較多	通常較少
業界偏好	✅ 大多用這個	需要低延遲時才用

實測對照 (範例 02 的真實輸出)

拍	din	狀態	Mealy	Moore	
3	1	S3	1	0	← Mealy 早一拍
4	1	S4	0	1	<<< 找到 1011 !

Mealy 在第 3 拍就拉高，Moore 在第 4 拍。同一份設計，兩種輸出並排看最清楚。

三、本週四個範例

```
cd week06_fsm
```

01_traffic_moore — 紅綠燈 ★

```
iverilog -o sim.out 01_traffic_moore.v 01_traffic_moore_tb.v
echo %ERRORLEVEL%
vvp sim.out
gtkwave 01_traffic_moore.gtkw
```

標準三段式 Moore 狀態機。RED(5拍) → GREEN(4拍) → YELLOW(2拍) 循環。

[!TIP] 在 GTKWave 看狀態：在 `state` 名字上按右鍵 → Data Format → Decimal 就會顯示 0/1/2 而不是 00 / 01 / 10，好認很多。

02_seq_detect — 序列偵測器 ★

```
iverilog -o sim.out 02_seq_detect.v 02_seq_detect_tb.v
echo %ERRORLEVEL%
vvp sim.out
gtkwave 02_seq_detect.gtkw
```

在資料流裡抓出 1011，Moore 和 Mealy 兩種輸出寫在同一個模組方便對照。

支援重疊偵測 (1011011 算兩次) —— 注意 S1011 的下一狀態怎麼設計的。

03_debounce — 按鍵去彈跳 ★★ 最實用

```
iverilog -o sim.out 03_debounce.v 03_debounce_tb.v
echo %ERRORLEVEL%
vvp sim.out
gtkwave 03_debounce.gtkw
```

實體按鍵按一下，電氣上會抖動幾十次。沒有去彈跳，你按一下計數器可能跳 5、6 下。

看波形對照三條線：

- `btn_raw` — 像鋸齒一樣亂抖

- `btn_clean` — 乾乾淨淨只有一次高低
- `btn_pulse` — 只在按下那一拍出現一個窄脈衝

實測：按 2 次 → 只產生 2 個脈衝 ✓

這個模組第 15 週接真實 FPGA 按鍵時會直接拿來用。

04_vending — 投幣式販賣機

```
iverilog -o sim.out 04_vending.v 04_vending_tb.v
echo %ERRORLEVEL%
vvp sim.out
gtkwave 04_vending.gtkw
```

飲料 25 元，可投 5 / 10 元，會找零。

實測：

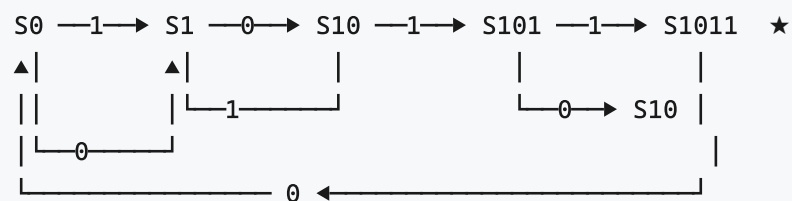
```
投 10 元 -> 累積 10 元
投 10 元 -> 累積 20 元
投 10 元 -> 出貨！找零 5 元
```

四、怎麼從「需求」寫出狀態機

步驟（照做就對了）：

1. 列出所有狀態 —— 問自己「這個電路需要記住哪些不同的情況？」
2. 畫狀態圖 —— 每個狀態畫一個圈，箭頭標上「什麼條件會轉過去」
3. 定義 `localparam` —— 每個狀態一個名字（不要用魔術數字）
4. 翻譯第 2 段 —— 一個狀態一個 `case` 分支，箭頭就是 `if`
5. 翻譯第 3 段 —— 每個狀態的輸出是什麼

以序列偵測器 `1011` 為例：



每個狀態代表「目前已經對上了幾個字元」。

五、練習題

練習 1

把 `01_traffic_moore` 加上「行人按鈕」：按下之後，綠燈最多再維持 2 拍就切黃燈。

練習 2 ★

寫一個偵測 `110` 的序列偵測器（Moore 版），並寫自我檢查測試（第 5 週學的）。

練習 3 ★

把 `04_vending` 改成也接受 50 元，飲料改成 65 元。

練習 4 (思考)

下面這個狀態機有什麼問題？

```

always @(*) begin
    case (state)
        S0: next_state = S1;
        S1: next_state = S2;
        S2: next_state = S0;
    endcase
end

```

► 看答案

六、本週檢核

- [] 四個範例都跑過，波形都看過
 - [] 能默寫三段式的骨架（不看筆記）
 - [] 02 的波形上指得出 Mealy 比 Moore 早一拍
 - [] 03 的波形上指得出 `btn_raw` 抖動、`btn_clean` 乾淨
 - [] 練習 2 寫出來且測試通過
 - [] 練習 4 兩個問題都答對
-

附註：關於 `sim` 這個指令

你可能在別的地方看過我寫的 `sim` 範例名稱。那是我自己做的批次檔（`sim.bat`），不是 Verilog 或 iverilog 的標準指令，教科書上查不到。

它做的事就是把上面那四行包起來、順便幫你檢查結束碼。用不用都可以，也可以刪掉。這份 README 裡給的全部都是原始指令。